Mathematical background for neural networks

1 Mathematical background

1.1 Introduction

Our neural network code is created with a modular design, where each layer of the network is an instance of its respective layer class, which inherits from the base class `Layer`. This allows for quick and easy creation and changes to the network architecture. To achieve this, however, we need to compute the loss gradient with repect to each layer regardless of the layer that came before it. We find that the chain rule tells us that all we need to compute the gradient of a given layer is the output gradient of that layer. We can show this by the equation,

$$\frac{\partial L}{\partial \mathbf{W}_i} = \frac{\partial L}{\partial \mathbf{Y}_i}\frac{\partial \mathbf{Y}_i}{\partial \mathbf{W}_i} \tag{1}$$

where L is the loss function, $\frac{\partial L}{\partial \mathbf{Y}_i}$ is the output gradient of layer i. Since for layer i, the output $\mathbf{Y}_i$ is equal to the input $\mathbf{X}_{i+1}$ of the next layer $i+1$. We also need to compute the input gradient with respect to each layer and return it for use of the previous layer, for which it becomes the output graident.

$$\frac{\partial L}{\partial \mathbf{Y}_i} = \frac{\partial L}{\partial \mathbf{X}_{i+1}} \tag{2}$$

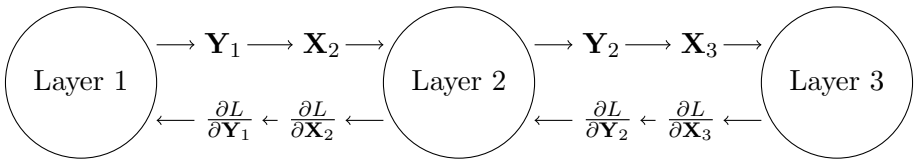

Figure 1: Forward and backward pass of a neural network

1.2 Fully Connected Layer

1.2.1 Forward Propagation

Our dense or fully connected layer forward propagation is defined by the equation,

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{B} \tag{3}$$

where $\mathbf{X}$ is the input, $\mathbf{W}$ is the weight matrix, $\mathbf{B}$ is the bias vector, and $\mathbf{Y}$ is the output. The weights matrix is of size $n \times m$ where n is the number of input features and m is the number of output features. The bias vector is of size m. If the input matrix is of size $n \times o$ where o is the

number of samples, then the output matrix is of size $m \times o$. We can write the i-th element of the b-th output vector in a batch of outputs as,

$$y_i^{[b]} = \sum_{j=1}^{n} w_{i,j} x_j^{[b]} + b_i \tag{4}$$

1.2.2 Loss Gradient with respect to the Weight Matrix

The error gradient with respect to the weigth matrix is given by,

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{W}} \tag{5}$$

. Within our backward propagation method we are given the output gradient $\frac{\partial L}{\partial \mathbf{Y}}$ as an argument. This leaves us to compute $\frac{\partial \mathbf{Y}}{\partial \mathbf{W}}$. For any particular element of the gradient $\frac{\partial \mathbf{Y}}{\partial \mathbf{W}}$ we can substitute equation 4 into the equation,

$$\frac{\partial y_t^{[b]}}{\partial w_{i,j}} = \frac{\partial}{\partial w_{t,j}} \left(\sum_{j=1}^{n} w_{i,j} x_j^{[b]} + b_i \right) = \begin{cases} x_j^{[b]} & \text{if } i = t \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

. Representing the gradient of the loss function with respect to the weight matrix as a matrix, we have,

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial L}{\partial w_{1,1}} & \frac{\partial L}{\partial w_{1,2}} & \cdots & \frac{\partial L}{\partial w_{1,n}} \\ \frac{\partial L}{\partial w_{2,1}} & \frac{\partial L}{\partial w_{2,2}} & \cdots & \frac{\partial L}{\partial w_{2,n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial w_{m,1}} & \frac{\partial L}{\partial w_{m,2}} & \cdots & \frac{\partial L}{\partial w_{m,n}} \end{bmatrix} \tag{7}$$

. It follows from equation 6 that the gradient of the loss function with respect a particular weight $w_{i,j}$ is given by,

$$\frac{\partial L}{\partial w_{i,j}} = \sum_{b=1}^{o} \frac{\partial L}{\partial y_i^{[b]}} x_j^{[b]} \tag{8}$$

. Substituting equation 8 into equation 7 we find the that final gradient of the loss function with respect to the weight matrix is given by,

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{Y}} \mathbf{X}^T \tag{9}$$

1.2.3 Loss Gradient with respect to the Bias Vector

The error gradient with respect to the bias vector is given by,

$$\frac{\partial L}{\partial \mathbf{B}} = \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{B}} \tag{10}$$

. Again, we are given the output gradient $\frac{\partial L}{\partial \mathbf{Y}}$ as an argument, so our task is to compute $\frac{\partial \mathbf{Y}}{\partial \mathbf{B}}$. Referencing equation 4, we can see that the gradient of the output with respect to the bias vector is given by,

$$\frac{\partial y_i^{[b]}}{\partial b_j} = \frac{\partial}{\partial b_j}\left(\sum_{j=1}^{n} w_{i,j} x_j^{[b]} + b_i\right) = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{11}$$

. Representing the gradient of the loss function with respect to the bias vector as a vector, we have,

$$\frac{\partial L}{\partial \mathbf{B}} = \begin{bmatrix} \frac{\partial L}{\partial b_1} \\ \frac{\partial L}{\partial b_2} \\ \vdots \\ \frac{\partial L}{\partial b_m} \end{bmatrix} \tag{12}$$

. It follows that the gradient of the loss function with respect to a particular bias b_i is given by,

$$\frac{\partial L}{\partial b_i} = \sum_{b=1}^{o} \frac{\partial L}{\partial y_i^{[b]}} \tag{13}$$

1.2.4 Loss Gradient with respect to the Input

1.3 Activation Functions

Activation functions are used to introduce non-linearity into our neural network. We used a total of four activation functions in our code, the ReLU, Sigmoid, Tanh, and Softmax functions. Since all activation functions are not trainable, we only need to compute the input gradient. The input gradient is given by,

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{X}} \tag{14}$$

. Hence, for each activation function we simply need to compute the derivative of the function with respect to the input.

1.3.1 ReLU

The ReLU function is defined as,

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{15}$$

Following equation 14, the input gradient is given by,

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} f'(\mathbf{X}) = \frac{\partial L}{\partial \mathbf{Y}} \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{16}$$

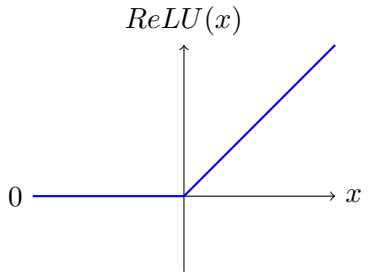

Figure 2: ReLU function

1.3.2 Sigmoid

The sigmoid function is defined as,

$$f(x) = \frac{1}{1 + e^{-x}} \tag{17}$$

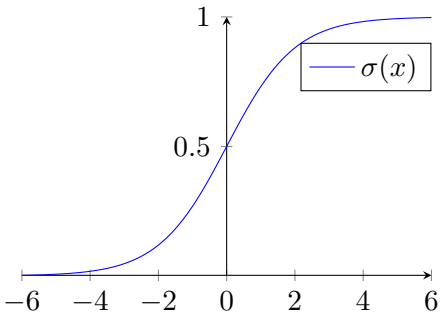

Figure 3: Sigmoid function

The derivative of the sigmoid function is given by,

$$f'(x) = f(x)(1 - f(x)) \tag{18}$$

Hence, the input gradient is given by,

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} f'(\mathbf{X}) = \frac{\partial L}{\partial \mathbf{Y}} f(\mathbf{X})(1 - f(\mathbf{X})) \tag{19}$$

1.3.3 Tanh

The tanh function is defined as,

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{20}$$

The derivative of the tanh function is given by,

$$f'(x) = 1 - f(x)^2 \tag{21}$$

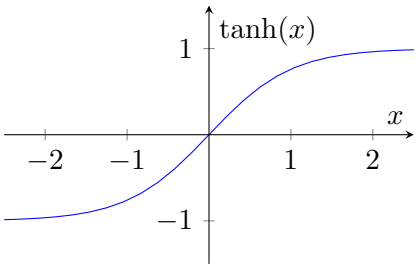

Figure 4: Tanh function

Hence, the input gradient is given by,

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} f'(\mathbf{X}) = \frac{\partial L}{\partial \mathbf{Y}} (1 - f(\mathbf{X})^2) \tag{22}$$

2 Convolutional Layer

3 Loss Functions